

Multilingual OCR & Machine Translation System

Complete Backend Source Code Compilation (Nepalese & Sinhalese to English)

1. app.py

Main Flask application routing and request control pipeline handling file uploads, user authentication, and PDF processing workflows.

```
from flask import Flask, render_template, request, redirect, url_for, session
import os
import tempfile
from pdf2image import convert_from_path
from ocr_module import extract_text
from translation_module import translate_text, last_translation_method
import translation_module
from database import add_user, check_user

app = Flask(__name__)
app.secret_key = "super_secret_key"

# ----- HOME -----
@app.route("/")
def home():
    return redirect(url_for("login"))

# ----- SIGNUP -----
@app.route("/signup", methods=["GET", "POST"])
def signup():
    if request.method == "POST":
        email = request.form.get("email")
        password = request.form.get("password")
        if not email or not password:
            return "Please fill all fields"
        if add_user(email, password):
            return redirect(url_for("login"))
        else:
            return "User already exists"
    return render_template("signup.html")

# ----- LOGIN -----
@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        email = request.form.get("email")
        password = request.form.get("password")
        if check_user(email, password):
            session["user"] = email
            return redirect(url_for("index"))
        else:
            return "Invalid email or password"
    return render_template("login.html")
```

```

# ----- LOGOUT -----
@app.route("/logout")
def logout():
    session.pop("user", None)
    return redirect(url_for("login"))

# ----- TRANSLATOR PAGE -----
@app.route("/index")
def index():
    if "user" not in session:
        return redirect(url_for("login"))
    return render_template("index.html")

# ----- TRANSLATE -----
@app.route("/translate", methods=["POST"])
def translate():
    if "user" not in session:
        return redirect(url_for("login"))

    if "file" not in request.files:
        return "No file uploaded"

    file = request.files["file"]
    src_lang = request.form.get("src_lang")

    if file.filename == "":
        return "No selected file"

    temp_dir = tempfile.mkdtemp()
    file_path = os.path.join(temp_dir, file.filename)
    file.save(file_path)

    images = []

    # ----- PDF Handling -----
    if file.filename.lower().endswith(".pdf"):
        try:
            poppler_path = r"C:\Users\PAVAN\AppData\Local\Microsoft\WinGet\Packages\oschwartz10612.Poppler_Microsoft.Winget.Source_8wekyb3d8bbwe\poppler-25.07.0\Library\bin"
            pages = convert_from_path(file_path, poppler_path=poppler_path, dpi=300)
        except Exception as e:
            return f"PDF Conversion Error: {str(e)}"

        for i, page in enumerate(pages):
            img_path = os.path.join(temp_dir, f"page_{i}.png")
            page.save(img_path, "PNG")
            images.append(img_path)
    else:

```

```

        images.append(file_path)

# ----- OCR -----
extracted_text = ""
for img in images:
    try:
        text = extract_text(img, src_lang)
        extracted_text += text + " "
    except Exception as e:
        return f"OCR Error: {str(e)}"

extracted_text = extracted_text.strip()
if not extracted_text:
    return "No text detected from file"

# ----- TRANSLATION -----
try:
    translated_text = translate_text(extracted_text, src_lang)
except Exception as e:
    return f"Translation Error: {str(e)}"

# ----- Read which method was used -----
method = translation_module.last_translation_method # "google" or "nllb"

return render_template(
    "result.html",
    extracted_text=extracted_text,
    translated_text=translated_text,
    translation_method=method
)

# ----- RUN -----
if __name__ == "__main__":
    app.run(debug=True)

```

2. database.py

SQLite configuration interface initializing database schemas and handling user authentication transactions securely.

```
import sqlite3

DB_NAME = "users.db"

# ♦ Create users table (run once)
def create_table():
    conn = sqlite3.connect(DB_NAME)
    cursor = conn.cursor()

    cursor.execute("""
        CREATE TABLE IF NOT EXISTS users (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            email TEXT UNIQUE NOT NULL,
            password TEXT NOT NULL
        )
    """)

    conn.commit()
    conn.close()

# ♦ SIGNUP: Add new user
def add_user(email, password):
    conn = sqlite3.connect(DB_NAME)
    cursor = conn.cursor()

    try:
        cursor.execute(
            "INSERT INTO users (email, password) VALUES (?, ?)",
            (email, password)
        )
        conn.commit()
        return True # signup success
    except sqlite3.IntegrityError:
        return False # email already exists
    finally:
        conn.close()

# ♦ LOGIN: Check user
def check_user(email, password):
    conn = sqlite3.connect(DB_NAME)
    cursor = conn.cursor()

    cursor.execute(
        "SELECT * FROM users WHERE email=? AND password=?",
        (email, password)
    )
```

```
)

user = cursor.fetchone()
conn.close()

return user is not None

# ♦ Run once to create table
if __name__ == "__main__":
    create_table()
    print("Database ready")
```

3. ocr_module.py

Image processing pipeline and Tesseract configuration script customized to protect fragile native characters for script extraction.

```
import pytesseract
import cv2
import numpy as np

pytesseract.pytesseract.tesseract_cmd = r"C:\Program Files\Tesseract-OCR\tesseract.exe"

# -----
# KEY INSIGHT for Devanagari / Sinhala OCR:
#
# These scripts have very thin connecting strokes (matras, conjuncts, viramas).
# Heavy preprocessing like sharpening kernels, aggressive thresholding, and
# morphological ops BREAK these strokes – turning क्ष into क + ् + ष etc.
# The golden rule: DO LESS, not more. Upscale + gentle contrast is enough.
# -----

def preprocess_image(img, src_lang):
    """
    Minimal preprocessing pipeline tuned for Devanagari and Sinhala scripts.
    Avoids operations that damage thin connecting strokes.
    """
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    h, w = gray.shape

    # Step 1: Upscale to at least 3000px wide for high glyph resolution
    # Devanagari needs more resolution than Latin – use 3x minimum
    if w < 2000:
        scale = 3000 / w
        gray = cv2.resize(
            gray, None, fx=scale, fy=scale,
            interpolation=cv2.INTER_LANCZOS4 # Lanczos preserves fine strokes better than
cubic
        )
    elif w < 3000:
        scale = 3000 / w
        gray = cv2.resize(gray, None, fx=scale, fy=scale, interpolation=cv2.INTER_LANCZOS4)

    # Step 2: Very light Gaussian blur to reduce JPEG/screenshot compression artifacts
    # Kernel (3,3) sigmaX=0 – minimal, just removes pixel-level noise
    # Do NOT use NlMeansDenoising here – it blurs matra strokes
    gray = cv2.GaussianBlur(gray, (3, 3), 0)

    # Step 3: CLAHE for contrast – gentle settings
    # clipLimit=1.5 (not 2.0+) to avoid over-enhancing noise near strokes
    clahe = cv2.createCLAHE(clipLimit=1.5, tileGridSize=(8, 8))
    gray = clahe.apply(gray)
```

```

# Step 4: For printed/digital text (screenshots, PDFs) – return grayscale directly
# Tesseract's internal binarization handles clean digital text better than
# our adaptive threshold which tends to break thin Devanagari strokes
# Only apply threshold for scanned/photographed documents
if src_lang in ("nep", "sin"):
    # Return grayscale – let Tesseract LSTM handle binarization internally
    return gray

# For English fallback: apply light threshold
_, binary = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
return binary

def extract_text(image_path, src_lang):
    """
    Extract text using Tesseract with language-appropriate settings.

    PSM modes:
    3 = Fully automatic (best for multi-paragraph documents)
    6 = Assume a single uniform block of text (best for clean single-column text)
    4 = Single column of text (good for narrow text blocks)

    We try PSM 6 first (most reliable for clean documents), then fall back
    to PSM 3 if the result looks too short.
    """
    img = cv2.imread(image_path)
    if img is None:
        raise ValueError(f"Could not load image: {image_path}")

    processed = preprocess_image(img, src_lang)

    if src_lang == "nep":
        lang_str = "nep+hin"  # hin traineddata fills gaps in nep coverage
    elif src_lang == "sin":
        lang_str = "sin"
    else:
        lang_str = "eng"

    # PSM 6: single uniform block – works best for clean digital/screenshot text
    config_psm6 = "--oem 1 --psm 6"
    # PSM 3: fully automatic – works best for multi-column / complex layouts
    config_psm3 = "--oem 1 --psm 3"

    text_psm6 = pytesseract.image_to_string(processed, lang=lang_str,
    config=config_psm6).strip()
    text_psm3 = pytesseract.image_to_string(processed, lang=lang_str,
    config=config_psm3).strip()

    # Pick the result with more content (more characters = better extraction)
    text = text_psm6 if len(text_psm6) >= len(text_psm3) else text_psm3

    # Post-OCR cleanup: remove lines that are clearly noise

```



```
lines = text.split('\n')
clean_lines = []
for line in lines:
    stripped = line.strip()
    # Skip blank lines and lines with only punctuation/symbols
    if not stripped:
        continue
    # Skip lines shorter than 3 chars (almost always OCR noise)
    if len(stripped) < 3:
        continue
    clean_lines.append(stripped)

return '\n'.join(clean_lines)
```

4. preprocess.py

Standalone utility function managing pixel upscaling, noise reductions via filters, contrast adjustments, and morphological operations.

```
import cv2
import numpy as np

def preprocess_image(image_path):
    """
    Standalone preprocessing utility.
    Returns a binary (thresholded) image ready for Tesseract OCR.

    This is imported by ocr_module.py – do not call Tesseract here.
    """
    img = cv2.imread(image_path)
    if img is None:
        raise ValueError(f"Could not read image at path: {image_path}")

    # Grayscale
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # Upscale if too small
    h, w = gray.shape
    if w < 1500:
        scale = 1500 / w
        gray = cv2.resize(gray, None, fx=scale, fy=scale, interpolation=cv2.INTER_CUBIC)

    # Denoise
    gray = cv2.fastNlMeansDenoising(gray, h=15, templateWindowSize=7, searchWindowSize=21)

    # Contrast enhancement
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
    gray = clahe.apply(gray)

    # Sharpening kernel
    kernel = np.array([[0, -1, 0],
                       [-1, 5, -1],
                       [0, -1, 0]])
    gray = cv2.filter2D(gray, -1, kernel)

    # Adaptive threshold
    binary = cv2.adaptiveThreshold(
        gray, 255,
        cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
        cv2.THRESH_BINARY,
        blockSize=31,
        C=10
    )

    # Morphological closing to reconnect broken strokes
```

```
kernel_morph = np.ones((2, 2), np.uint8)
binary = cv2.morphologyEx(binary, cv2.MORPH_CLOSE, kernel_morph)

return binary
```

5. translation_module.py

Hybrid engine connecting cloud translation endpoints as primary provider with a transformer-backed localized model fallback.

```
# translation_module.py
# Hybrid Translation: Google Translate (online) + NLLB-200 600M (offline fallback)

import re
import socket

# — Google Translate —————
try:
    from deep_translator import GoogleTranslator
    GOOGLE_AVAILABLE = True
except ImportError:
    GOOGLE_AVAILABLE = False
    print("deep-translator not installed. Run: pip install deep-translator")

# — NLLB Offline Fallback —————
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
import torch

MODEL_NAME = "facebook/nllb-200-distilled-600M"
print(f"Loading offline fallback model: {MODEL_NAME} ...")
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
model = AutoModelForSeq2SeqLM.from_pretrained(MODEL_NAME)
device = "cuda" if torch.cuda.is_available() else "cpu"
model.to(device)
model.eval()
print(f"Offline model loaded on {device}")

# — Language code maps —————
# Google Translate BCP-47 codes
GOOGLE_LANG_MAP = {
    "nep": "ne", # Nepali
    "sin": "si", # Sinhala
    "eng": "en",
}

# NLLB BCP-47 codes
NLLB_LANG_MAP = {
    "nep": "npi_Deva",
    "sin": "sin_Sinh",
    "eng": "eng_Latn",
}
TARGET_LANG = "eng_Latn"

# Tracks which method was used last – read by app.py to show indicator
last_translation_method = "nllb" # "google" or "nllb"

# — Pre-translation substitution map (used only for NLLB offline fallback) ———
```

Google Translate doesn't need these – it handles all words correctly.

```
NEPALI_SUBSTITUTIONS = [  
    ("गोहीकहाँ", "मगरमच्छकहाँ"),  
    ("गोहीलाई", "मगरमच्छलाई"),  
    ("गोहीसँग", "मगरमच्छसँग"),  
    ("गोहीले", "मगरमच्छले"),  
    ("गोहीको", "मगरमच्छको"),  
    ("गोहीमा", "मगरमच्छमा"),  
    ("गोही", "मगरमच्छ"),  
    ("वाँदरले", "बाँदरले"),  
    ("वाँदरको", "बाँदरको"),  
    ("वाँदरलाई", "बाँदरलाई"),  
    ("वाँदर", "बाँदर"),  
    ("ढाडमा", "पिठ्युँमा"),  
    ("ढाडको", "पिठ्युँको"),  
    ("ढाड", "पिठ्युँ"),  
    ("मुटुको", "हृदयको"),  
    ("मुटुलाई", "हृदयलाई"),  
    ("मुटु", "हृदय"),  
    ("श्रीमतीले", "पत्नीले"),  
    ("श्रीमतीलाई", "पत्नीलाई"),  
    ("श्रीमतीको", "पत्नीको"),  
    ("श्रीमती", "पत्नी"),  
    ("श्रीमानले", "पतिले"),  
    ("श्रीमानको", "पतिको"),  
    ("श्रीमान", "पति"),  
    ("झस्कियो", "चकित भयो"),  
    ("मेरो चर", "मेरो घर"),  
]
```

```
SINHALA_SUBSTITUTIONS = [  
    ("කිඹුලාගේ", "මගරාගේ"),  
    ("කිඹුලාට", "මගරාට"),  
    ("කිඹුලා", "මගරා"),  
    ("ජම්බු ගෙඩි", "රෝස් ඇපල් ගෙඩි"),  
    ("ජම්බු", "රෝස් ඇපල්"),  
    ("ගොඩබිමට", "වෙරළට"),  
    ("ගොඩබිමේ", "වෙරළේ"),  
    ("ගොඩබිම", "වෙරළ"),  
    ("හදාවන", "හෘදය"),  
    ("හදවතේ", "හෘදයේ"),  
    ("හදවන", "හෘදය"),  
]
```

```
SUBSTITUTION_MAP = {  
    "nep": NEPALI_SUBSTITUTIONS,  
    "sin": SINHALA_SUBSTITUTIONS,  
}
```

```
# =====  
# UTILITY FUNCTIONS
```

```
# =====

def is_internet_available():
    """Check internet by connecting to Google's DNS (8.8.8.8)."""
    try:
        socket.setdefaulttimeout(3)
        socket.socket(socket.AF_INET, socket.SOCK_STREAM).connect(("8.8.8.8", 53))
        return True
    except Exception:
        return False

def clean_text(text):
    """Remove OCR artifacts and normalize whitespace."""
    text = re.sub(r'[\S\n]+', ' ', text) # normalize spaces
    text = re.sub(r'\n{3,}', '\n\n', text) # max 2 newlines
    text = re.sub(r'[|}{<>@™]+', '', text) # OCR artifacts (NO space here)
    lines = text.split('\n')
    clean_lines = [
        l.strip() for l in lines
        if l.strip() and not re.fullmatch(
            r'[\d\s\.\,\-\_\|\!\\?\/\\\@#$$%^&*()]+', l.strip()
        )
    ]
    return ' '.join(clean_lines).strip()

def apply_substitutions(text, src_lang):
    """Replace NLLB problem words with better synonyms (offline fallback only)."""
    for wrong, correct in SUBSTITUTION_MAP.get(src_lang, []):
        text = text.replace(wrong, correct)
    return text

def split_into_chunks(text, max_chars=4500):
    """
    Split text into chunks under max_chars.
    Google Translate has a 5000 char limit per request – stay under it.
    Splits on sentence boundaries to preserve meaning.
    """
    sentences = re.split(r'(?<=[.!?\u00DD\u00DF4])\s+', text)
    sentences = [s.strip() for s in sentences if s.strip()]

    chunks = []
    current = []
    current_len = 0

    for sentence in sentences:
        if current_len + len(sentence) > max_chars and current:
            chunks.append(' '.join(current))
            current = [sentence]
            current_len = len(sentence)

```

```

        else:
            current.append(sentence)
            current_len += len(sentence)

    if current:
        chunks.append(' '.join(current))

    return chunks

# =====
# GOOGLE TRANSLATE (PRIMARY – ONLINE)
# =====

def translate_with_google(text, src_lang):
    """
    Translate using Google Translate via deep-translator.
    Handles texts longer than 5000 chars by splitting into chunks.
    """
    src_code = GOOGLE_LANG_MAP.get(src_lang, "auto")
    chunks = split_into_chunks(text, max_chars=4500)

    translated_parts = []
    for chunk in chunks:
        try:
            result = GoogleTranslator(source=src_code, target="en").translate(chunk)
            if result and result.strip():
                translated_parts.append(result.strip())
            else:
                # Empty result – fall back to NLLB for this chunk
                translated_parts.append(translate_chunk_nllb(chunk, src_lang))
        except Exception as e:
            print(f"Google Translate error: {e} – using NLLB for this chunk")
            translated_parts.append(translate_chunk_nllb(chunk, src_lang))

    return ' '.join(translated_parts)

# =====
# NLLB OFFLINE FALLBACK
# =====

def get_nllb_bos_token_id():
    """Reliably get the eng_Latn token ID."""
    try:
        tid = tokenizer.convert_tokens_to_ids(TARGET_LANG)
        if tid == tokenizer.unk_token_id:
            raise ValueError("unk")
        return tid
    except Exception:
        return tokenizer.lang_code_to_id[TARGET_LANG]

```

```

def translate_chunk_nllb(text, src_lang):
    """Translate a single chunk using NLLB offline model."""
    src_code = NLLB_LANG_MAP.get(src_lang, "npi_Deva")
    tokenizer.src_lang = src_code
    forced_bos = get_nllb_bos_token_id()

    inputs = tokenizer(
        text,
        return_tensors="pt",
        truncation=True,
        max_length=512
    ).to(device)

    with torch.no_grad():
        tokens = model.generate(
            **inputs,
            forced_bos_token_id=forced_bos,
            max_new_tokens=400,
            num_beams=5,
            length_penalty=1.0,
            early_stopping=True,
            no_repeat_ngram_size=3,
            repetition_penalty=1.2,
        )

    return tokenizer.decode(tokens[0], skip_special_tokens=True).strip()

def translate_with_nllb(text, src_lang):
    """
    Full NLLB translation pipeline with substitutions and chunking.
    Used as offline fallback.
    """
    # Apply substitution map to fix known NLLB errors
    text = apply_substitutions(text, src_lang)

    sentences = re.split(r'(?<=[.!?\u00DDF\u00DF4])\s+', text)
    sentences = [s.strip() for s in sentences if s.strip()]

    # Build chunks of max 100 words with 2-sentence context window
    chunks = []
    i = 0
    while i < len(sentences):
        main, wc = [], 0
        j = i
        while j < len(sentences):
            w = sentences[j].split()
            if wc + len(w) > 100 and main:
                break
            main.append(sentences[j])
            wc += len(w)

```



```

        j += 1
        context = sentences[max(0, i - 2):i]
        chunks.append({
            "context": ' '.join(context),
            "main": ' '.join(main)
        })
        i = j

translated_parts = []
for idx, chunk in enumerate(chunks):
    try:
        full_input = (chunk["context"] + " " + chunk["main"]).strip() \
            if chunk["context"] else chunk["main"]
        result = translate_chunk_nllb(full_input, src_lang)
        translated_parts.append(result)
    except Exception as e:
        translated_parts.append(f"[Segment {idx+1} error: {str(e)}]")

return ' '.join(translated_parts)

# =====
# MAIN ENTRY POINT - called from app.py
# =====

def translate_text(text, src_lang):
    """
    Hybrid translation:
    1. Clean OCR text
    2. Try Google Translate (if internet available)
    3. Fall back to NLLB-200 600M if offline or Google fails

    Args:
        text : OCR-extracted source text
        src_lang : "nep" or "sin"

    Returns:
        Translated English string
    """
    if not text or not text.strip():
        return "[No text to translate]"

    if src_lang not in ("nep", "sin", "eng"):
        raise ValueError(f"Unsupported language: '{src_lang}'. Use 'nep' or 'sin'.")

    # Step 1: Clean OCR artifacts
    cleaned = clean_text(text)
    if not cleaned:
        return "[Text was empty after cleaning]"

    global last_translation_method

```

```

# Step 2: Try Google Translate (online)
if GOOGLE_AVAILABLE and is_internet_available():
    print("Internet detected – using Google Translate")
    try:
        result = translate_with_google(cleaned, src_lang)
        if result and result.strip():
            last_translation_method = "google"
            return result
    except Exception as e:
        print(f"Google Translate failed: {e} – switching to NLLB")

# Step 3: Offline fallback – NLLB 600M
print("Using offline NLLB model")
last_translation_method = "nllb"
return translate_with_nllb(cleaned, src_lang)

```

6. download_model.py

Setup dependency script to pre-fetch and load language model weights locally from Hugging Face repositories.

```

from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

model_name = "facebook/nllb-200-distilled-600M"

print("Downloading tokenizer...")
tokenizer = AutoTokenizer.from_pretrained(model_name)

print("Downloading model...")
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

print("Model downloaded successfully!")

```